

GRU

A small recurrent network over the RR sequence plus the engineered features.

A 16-unit GRU over the 30-beat sequence. Accurate in float but rejected because int8 quantisation costs it 1.6-3.6 ms.

Architecture

- GRU(16, unrolled for a loop-free TFLite graph) over the (30, 2) sequence; its last state is concatenated with the 9 features, then Dense(32, ReLU) - Dense(1).
- 1825 parameters, 1696 MACs per beat (stateful streaming), about 1.8 KB int8.

Layer table of gru (B = batch)

layer	configuration	output shape	params
InputLayer		(B, 30, 2)	0
GRU	units=16, activation=tanh, unroll=True	(B, 16)	960
InputLayer		(B, 9)	0
Concatenate		(B, 25)	0
Dense	units=32, activation=relu	(B, 32)	832
Dense	units=1	(B, 1)	33

Inputs and feature engineering (gru)

- Sequence input** (TCN/GRU): the last 30 RR intervals as two channels, z-RR and z-dRR (first step dRR = 0), z-scored with mean/std fitted on the training subjects only (zero variance -> std 1).
- Tabular input**: the engineered features of the newest beat (see 01_feature_engineering.pdf), z-scored per feature with training statistics.
- Target**: $dRR[n+1] = RR[n+1] - RR[n]$. Residual models predict the z-scored correction over the RLS forecast (dRR - rls_pred); the output is de-normalised and added back to rls_pred.

Tabular features (9):

feature	definition
rmssd	root mean square of the 29 successive differences
rs	rescaled range: (max - min of the cumulative sum of RR - mean) / std
ccm	complex correlation measure: Poincare-triangle area / (pi SD1 SD2 (W-3)), clipped to [0, 1]
guzik	Guzik asymmetry: sum of positive dRR / sum of dRR
rls_pred	RLS AR(8) forecast of the next dRR (ms), clamped to +-300
rls_err	last RLS innovation: true dRR minus the previous forecast (ms)
bp	RSA band-pass output: RBJ biquad on raw RR, centre 0.30 cycles/beat, Q 1
bp_prev	previous band-pass output (gives the phase with bp)
run_len	signed length of the current run of same-sign dRR (+ = decelerating)

Training

Keras, AdamW (weight decay 1e-4), Huber loss (delta 1, z units), batch 512, steps_per_execution 64, epochs of 2M random training windows (about 10% of the 20.8M), EarlyStopping on val_loss (restore best weights) and ReduceLROnPlateau (factor 0.5, min 1e-5), seed 211. int8: full-integer post-training quantisation (TFLite, int8 in/out), deterministic best-of-K calibration (8 draws of 2k/5k training windows; the draw with the smallest gap on held-out training subjects is kept).

Variants trained

run	change vs parent	feature set (n)	params	MACs/beat	training	epochs
gru	same inputs as the TCNs	seq (9)	1825	1696	from scratch, lr 0.001	49

Results

model	val open loop	closed-lo op model	10%	30%	50%	80%	RSA invented at 80%	test whole-series 80%	val source
gru	33.90	int8	28.04	31.38	35.34	45.07	12%	43.84	round 2 (gru)

RMSE in ms. **val open loop**: one-step-ahead on the true history, 16 validation subjects. **10-80%**: closed loop (pacemaker use) on validation: that share of beats dropped at random from beat 0, each filled by the model and fed back; RMSE on model-filled beats, first 5000 beats per subject, mean of the seeds of that round (3 seeds from round 2 on). int8 is used where it exists. **RSA invented**: share of subject-runs whose filled series gains an RSA peak the original lacks. **test whole-series 80%**: diagnostic on the 10 test subjects (whole series, seed 7, results/test/ folder); it was not used for any selection.

Correlation between the original and the filled series (validation and test subjects)

model	corr RR validation	corr RR test	corr dRR validation	corr dRR test	RMSE filled, validation (ms)	RMSE filled, test (ms)
gru	0.9352	0.9397	0.266	0.263	50.24	43.84

Computed on the 16 validation and the 10 test subjects (results/validation/ and results/test/ folders), on the ENTIRE series, after dropping 80% of the beats (seed 7, from beat 0) and filling them in closed loop with the model (Burg-AR fill for the first 30 beats): **corr RR** = `np.corrcoef(original_series, modified_series)[0, 1]`; **corr dRR** = the same function on the beat-to-beat changes, `np.corrcoef(np.diff(original), np.diff(filled))[0, 1]`, which isolates the short-term dynamics. Mean over subjects; per-subject values in <folder>/crosscorr.csv. **RMSE filled**: RMSE on the model-filled beats of the same runs. Diagnostic: the selection used the round-4 protocol, not these runs.

Verdict

- Float: ties micro_tcn at every drop rate with 19% fewer MACs and beats mlp in 15/16 subjects.
- int8: gap +2.71% on validation (over the 2% limit) and 1.6-3.6 ms lost in the closed loop; it also invents an RSA peak at 50% on one subject: rejected. Quantising recurrent state is known to be hard on MCUs.

Abbreviations and acronyms used in this document

term	meaning
AdamW	Adam optimiser with decoupled weight decay (the training algorithm of the neural models)
AR	autoregressive: a value predicted as a weighted sum of past values; AR(p) uses p past values
B	batch dimension (number of windows processed together) in the layer output shapes
bp	feature: RSA band-pass output: RBJ biquad on raw RR, centre 0.30 cycles/beat, Q 1
bp_prev	feature: previous band-pass output (gives the phase with bp)
Burg-AR	AR model whose coefficients were fitted with Burg's method (used for the first 30 beats)
ccm	complex correlation measure: non-linearity index of the Poincare plot of consecutive RR intervals
dRR	beat-to-beat change of the RR interval: $dRR[n+1] = RR[n+1] - RR[n]$ (ms); the quantity predicted
GRU	gated recurrent unit (a recurrent neural-network cell)
gru	gated recurrent unit (run name)
guzik	feature: Guzik asymmetry: sum of positive dRR / sum of dRR
HRV	heart rate variability
int8	8-bit integer: weights and activations quantised to 8-bit integers, as run on the microcontroller; _int8 = the int8 version of a run
KB	kilobyte (1,024 bytes)
lr	learning rate (step size of the optimiser)
MAC	multiply-accumulate operation (one multiplication plus one addition): the unit of compute cost on the microcontroller; MACs/beat = how many the model needs per heartbeat
MACs	multiply-accumulate operations (see MAC)
MCU	microcontroller unit
mip	multilayer perceptron (run name)
ms	milliseconds
np.corrcoef	NumPy's Pearson correlation-coefficient function
np.diff	NumPy's successive-difference function
Q	quality factor of a band-pass filter: centre frequency / bandwidth
RBJ	Robert Bristow-Johnson's standard biquad (second-order) filter formulas
ReLU	rectified linear unit, $\max(0, x)$ (an activation function)
RLS	recursive least squares: an adaptive filter that updates its coefficients at every beat
rls_err	feature: last RLS innovation: true dRR minus the previous forecast (ms)
rls_pred	feature: RLS AR(8) forecast of the next dRR (ms), clamped to ± 300
RMSE	root mean square error (ms): the main accuracy measure, lower is better
rmssd	root mean square of successive differences of the RR intervals in the window
RR	RR interval: time between two consecutive heartbeats (R waves of the ECG), in ms
rs	rescaled range: $(\max - \min \text{ of the cumulative deviations from the mean }) / \text{standard deviation}$
RSA	respiratory sinus arrhythmia: the oscillation of the heart rate driven by breathing
run_len	feature: signed length of the current run of same-sign dRR (+ = decelerating)
SD1	Poincare-plot standard deviation across the identity line (short-term variability)
SD2	Poincare-plot standard deviation along the identity line (long-term variability)
seq	feature-set / model-kind name: sequence model (takes the raw RR window plus tabular features)
TCN	temporal convolutional network (causal, dilated 1-D convolutions)
tcn	temporal convolutional network (run-name part)
TFLite	TensorFlow Lite: the runtime and file format for models on embedded devices
val	validation: the 16 validation subjects, used for model selection
W	window length: the 30 most recent RR intervals
z	z-score: $(\text{value} - \text{mean}) / \text{standard deviation}$, with statistics from the training subjects